QualiAdept

Workbook Complet: Sesiunea 4 - Manipularea DOM-ului

Partea 1: Codul Aplicației (Simulator QA) 2
Partea 2: Mega-Suită de Exerciții (Nivel Intermediar) 5
Secțiunea A: Selectoare și Proprietăți (DOM Retrieval) 5
Exercițiul 1: Numărarea elementelor adăugate dinamic (querySelectorAll) 6
Exercițiul 2: Găsirea elementelor după atribute de test (Stil Playwright) 7
Exercițiul 3: Extragerea textelor (Verificare Assertions) 8
Secțiunea B: Evenimente și Interactivitate (Event Listeners) 9
Exercițiul 4: Evenimentul de tastatură (keydown) 9
Exercițiul 5: Manipularea live a claselor (classList) 10
Exercițiul 6: Prevenirea acțiunilor nedorite (preventDefault) 11
Secțiunea C: Manipulare Dinamică a DOM-ului 12
Exercițiul 7: Inserarea unui lot de date deodată 12
Exercițiul 8: Adăugarea unui indicator de "Test Picat" 13
Secțiunea D: Latență și Propagare Avansată (Bubbling) 14
Exercițiul 9: Latența Distructivă (Testarea Flaky) 14
Exercițiul 10: Event Delegation Peste Tot - Bulele de Săpun (Status "Passed") 16
Partea 3: Nivelul "Masterclass" - Exerciții Avansate pentru QA 17
Exercițiul 11: Colectarea datelor într-un Array real (Scraping) 17
Exercițiul 12: Navigarea Relativă în DOM (Element Matching) 18
Exercițiul 13: Declanșarea Programatică a Evenimentelor (Bypass) 20
Exercițiul 14: Simulare Checkbox "Terms & Conditions" 21
Exercițiul 15: Validarea vizibilității (Computed Style) 23

Acest document conține aplicația web de bază pentru exersare (Simulatorul QA) și o suită completă de 15 exerciții practice (împărțite pe niveluri de dificultate) pentru a aprofunda selectoarele, evenimentele, manipularea DOM-ului, latența (setTimeout) și propagarea evenimentelor.

Partea 1: Codul Aplicației (Simulator QA)

Acesta este codul complet al aplicației. Salvează-l într-un fișier cu extensia .html (ex: index.html), apoi deschide-l cu orice browser (Chrome, Firefox, Edge).

```
<!DOCTYPE html>
<html lang="ro">
<head>
    <meta charset="UTF-8">
    <title>Simulator QA - Manipularea DOM-ului</title>
    <style>
        body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-
serif; background-color: #f4f4f9; padding: 20px; }
        .card { background: white; padding: 20px; border-radius: 8px; box-
shadow: 0 4px 6px rgba(0,0,0,0.1); max-width: 500px; margin: auto; }
        .form-group { margin-bottom: 15px; }
        input[type="text"] { width: 100%; padding: 10px; box-sizing:
border-box; border: 1px solid #ccc; border-radius: 4px; }
        button { background: #007bff; color: white; border: none; padding:
10px 15px; cursor: pointer; border-radius: 4px; transition: 0.3s;}
        button:disabled { background: #ccc; cursor: not-allowed; }
        ul { list-style-type: none; padding: 0; }
        li { background: #e9ecef; margin-bottom: 8px; padding: 12px;
display: flex; justify-content: space-between; align-items: center; border-
radius: 4px; }
        .delete-btn { background: #dc3545; padding: 5px 10px; font-size:
12px; }
        .loading { display: none; color: #ffc107; font-weight: bold;
margin-top: 10px; font-size: 14px; }
        .completed { text-decoration: line-through; color: #6c757d; }
```

```
    </style>
</head>
<body>
    <div class="card">
        <h2>QA Event & DOM Simulator</h2>
        <form id="qa-form">
            <div class="form-group">
                <label for="task-input">Adaugă Scenariu de
Test:</label><br><br>
                <input type="text" id="task-input" placeholder="Ex: Test
login valid...">
                <small id="char-count" style="color: gray; display: block;
margin-top: 5px;">0 caractere introduse</small>
            </div>
            <!-- Butonul pornește dezactivat -->
            <button type="submit" id="submit-btn" data-testid="save-btn"
disabled>Salvează (Latență 2s)</button>
            <p id="loading-msg" class="loading">Se comunică cu
serverul...</p>
        </form>
        <hr>
        <h3>Scenarii Active</h3>
        <ul id="task-list">
            <!-- Elementele vor fi adăugate aici de JS -->
        </ul>
    </div>

    <script>
        // 1. Selectarea Elementelor (Obiectul document)
        const form = document.querySelector('#qa-form');
        const input = document.querySelector('#task-input');
        const submitBtn = document.querySelector('#submit-btn');
        const taskList = document.querySelector('#task-list');
        const charCount = document.querySelector('#char-count');
        const loadingMsg = document.querySelector('#loading-msg');

        // 2. Ascultător pentru Evenimentul 'input' - Validare Live
        input.addEventListener('input', () => {
            const text = input.value.trim();
            charCount.textContent = `${text.length} caractere introduse`;

            if (text.length === 0) {
                submitBtn.disabled = true;
```

```
            } else {
                submitBtn.disabled = false;
            }
        });

        // 3. Ascultător pentru Evenimentul 'submit' - Latență & Creare
Dinamică
        form.addEventListener('submit', (e) => {
            e.preventDefault();
            const taskText = input.value.trim();

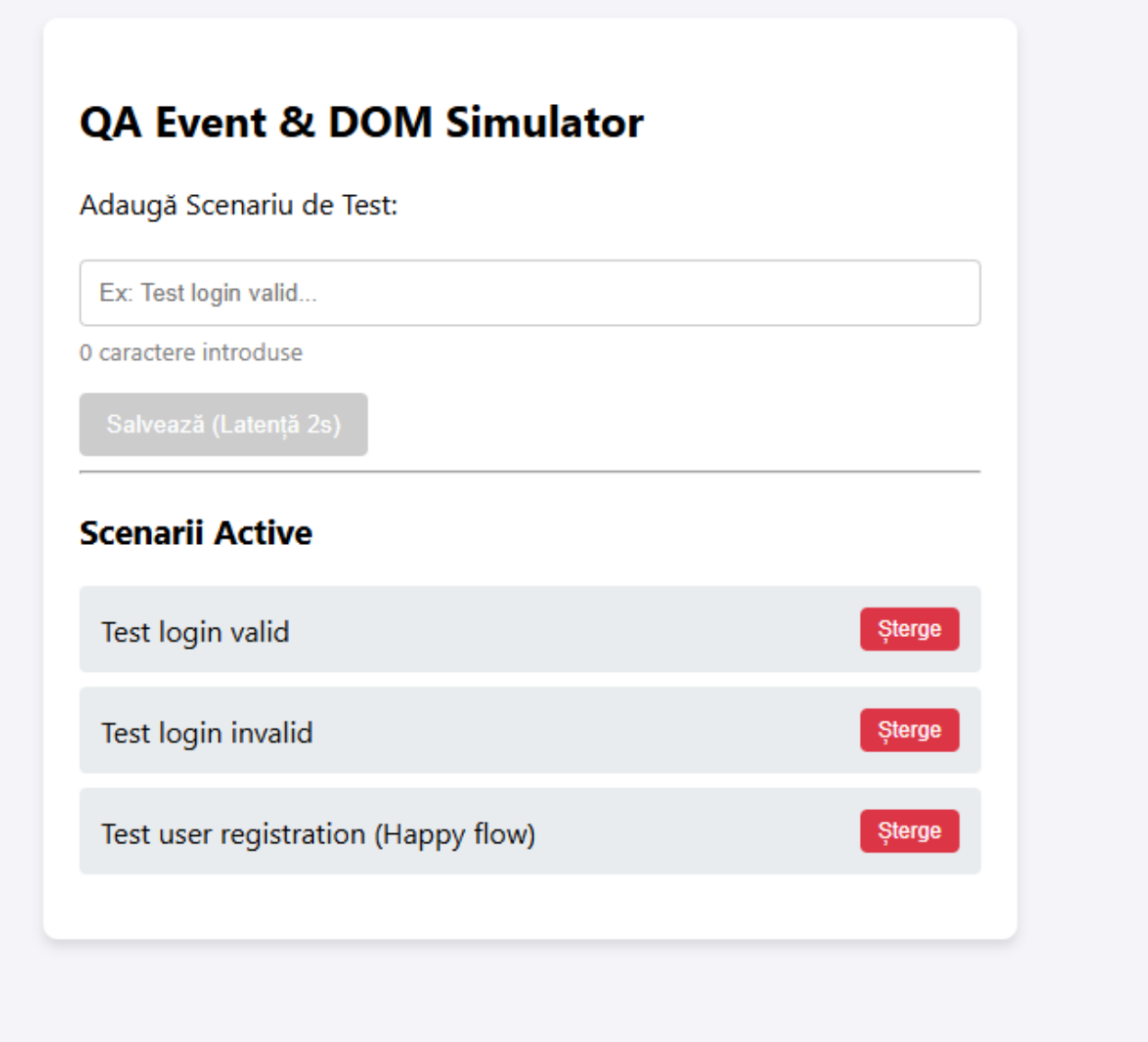
            submitBtn.disabled = true;
            loadingMsg.style.display = 'block';

            setTimeout(() => {
                const li = document.createElement('li');
                li.className = 'task-item';
                li.innerHTML = `<span class="task-text">${taskText}</span>
<button class="delete-btn" data-testid="del-btn">Șterge</button>`;

                taskList.appendChild(li);

                input.value = '';
                charCount.textContent = '0 caractere introduse';
                loadingMsg.style.display = 'none';
            }, 2000);
        });

        // 4. Ștergerea Dinamică (Event Delegation / Bubbling)
        taskList.addEventListener('click', (e) => {
            if (e.target.classList.contains('delete-btn')) {
                e.target.closest('li').remove();
            }
        });
    </script>
</body>
</html>
```



Partea 2: Mega-Suită de Exerciții (Nivel Intermediar)

Aceste 10 exerciții se rezolvă direct în consola browserului (F12 -> Console) după ce ai deschis aplicația de mai sus.

Secțiunea A: Selectoare și Proprietăți

(DOM Retrieval)

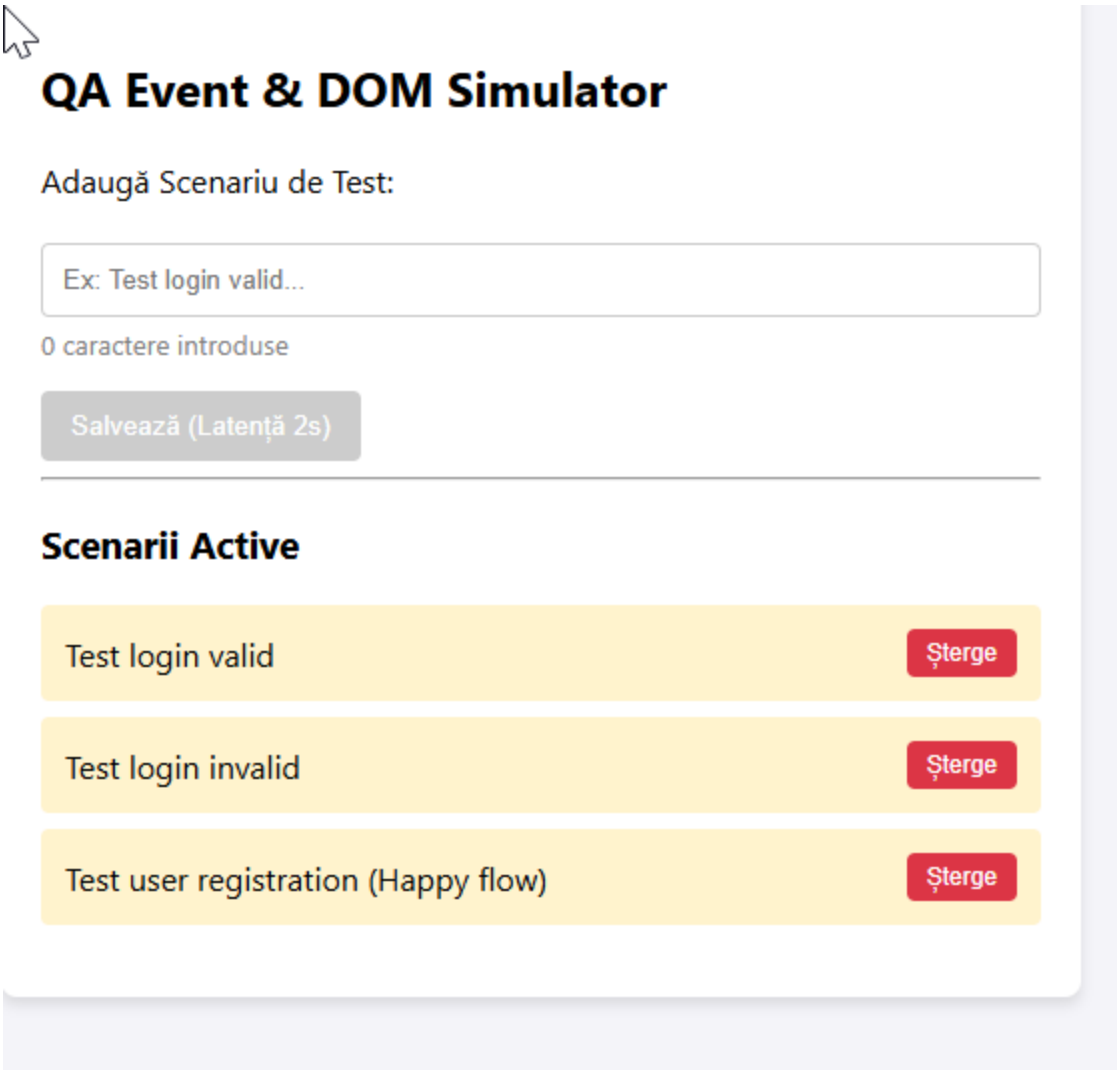
Exercițiul 1: Numărarea elementelor adăugate dinamic (querySelectorAll)

Scenariu: Trebuie să verifici de câte ori s-a randat clasa .task-item pe ecran.

🤔 ❓ Cerință: Adaugă manual 3 task-uri. Scrie un script care găsește toate elementele cu clasa task-item și le face fundalul galben.

☑ **Soluție**:

```
const toateTaskurile = document.querySelectorAll('.task-item');
for (let i = 0; i < toateTaskurile.length; i++) {
    toateTaskurile[i].style.backgroundColor = "#fff3cd";
}
```



Exercițiul 2: Găsirea elementelor după atribute de test (Stil Playwright)

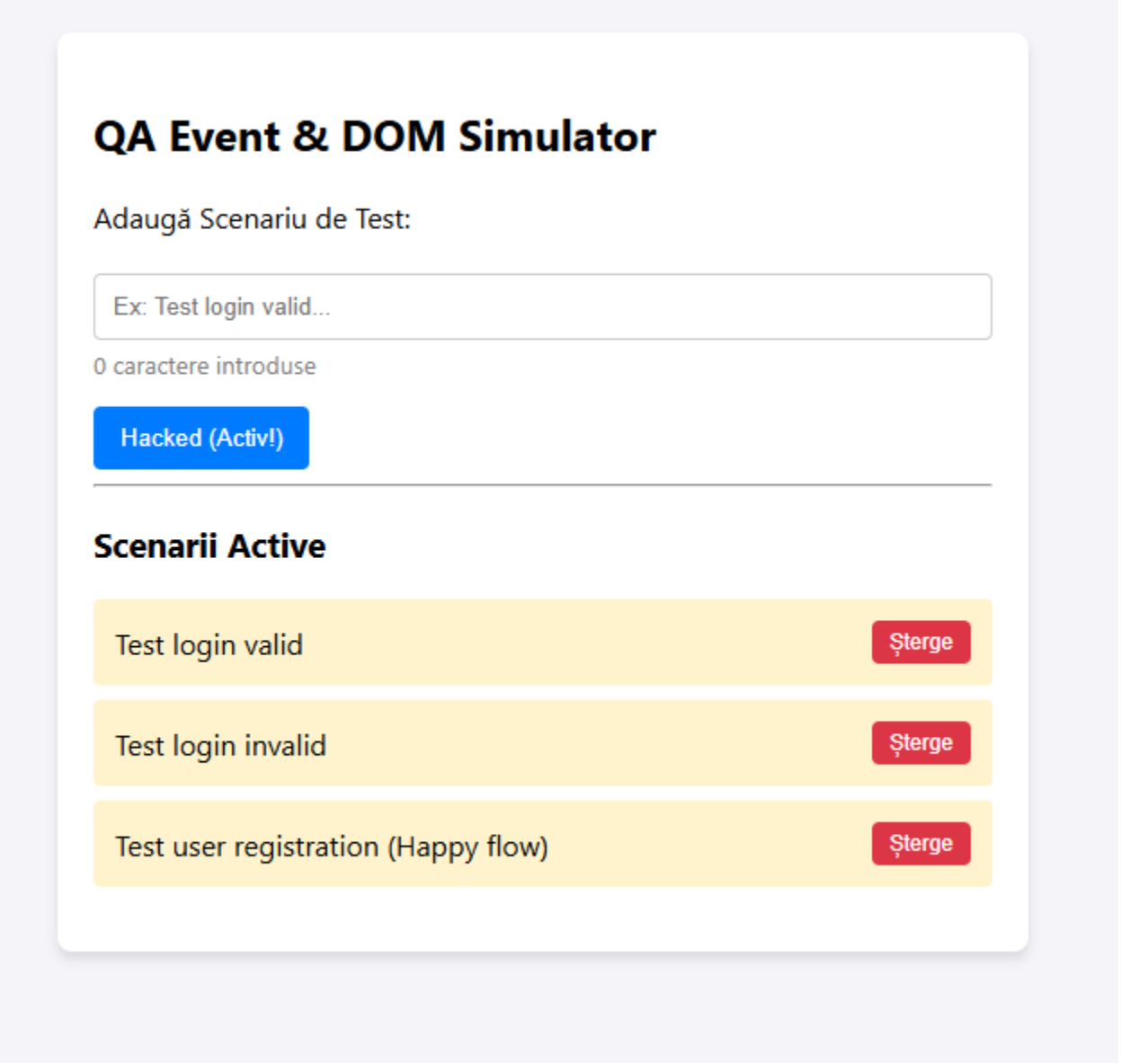
Scenariu: Butonul de submit are ID-ul #submit-btn, dar inginerii de QA preferă atributul data-testid="save-btn".

🤓 ❓ **Cerință**: Selectează butonul "Salvează" folosind strict atributul data-testid și forțează-l să devină activ (disabled = false), chiar dacă input-ul este gol.

☑ **Soluție**:

```
const btnSalveaza = document.querySelector('[data-testid="save-btn"]');
```

```
btnSalveaza.disabled = false;
btnSalveaza.textContent = "Hacked (Activ!)";
```



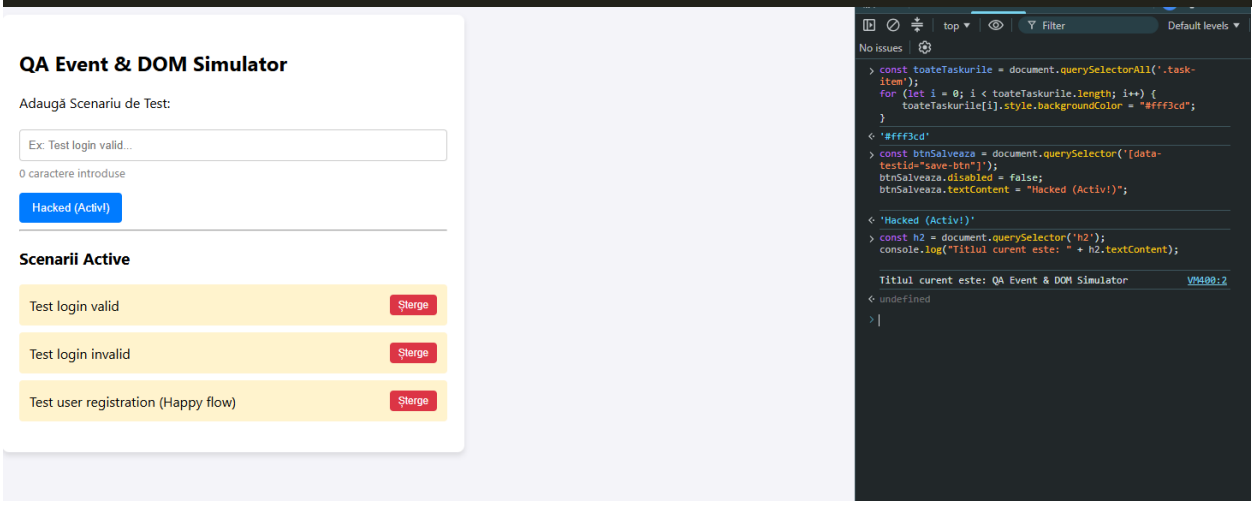
Exercițiul 3: Extragerea textelor (Verificare Assertions)

Scenariu: Vrei să te asiguri că titlul aplicației este exact cel așteptat de business.

🕵️ ❓ Cerință: Găsește elementul <h2> și printează în consolă textul său folosind proprietatea textContent.

☑ **Soluție**:

```
const h2 = document.querySelector('h2');
console.log("Titlul curent este: " + h2.textContent);
```



Secțiunea B: Evenimente și Interactivitate (Event Listeners)

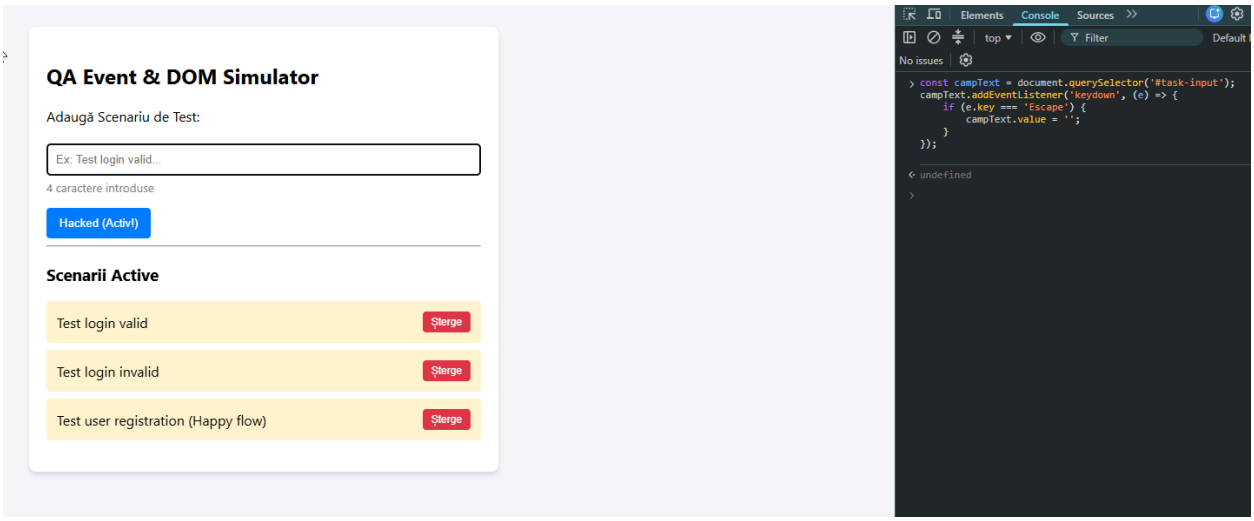
Exercițiul 4: Evenimentul de tastatură (keydown)

Scenariu: Vrem ca utilizatorul să poată goli câmpul de text instantaneu apăsând tasta "Escape".

🤓 ❓ **Cerință**: Găsește input-ul. Adaugă un ascultător pentru keydown. Dacă proprietatea e.key este "Escape", setează valoarea input-ului la un string gol "".

☑ **Soluție**:

```
const campText = document.querySelector('#task-input');
campText.addEventListener('keydown', (e) => {
    if (e.key === 'Escape') {
        campText.value = '';
    }
});
```



Exercițiul 5: Manipularea live a claselor (classList)

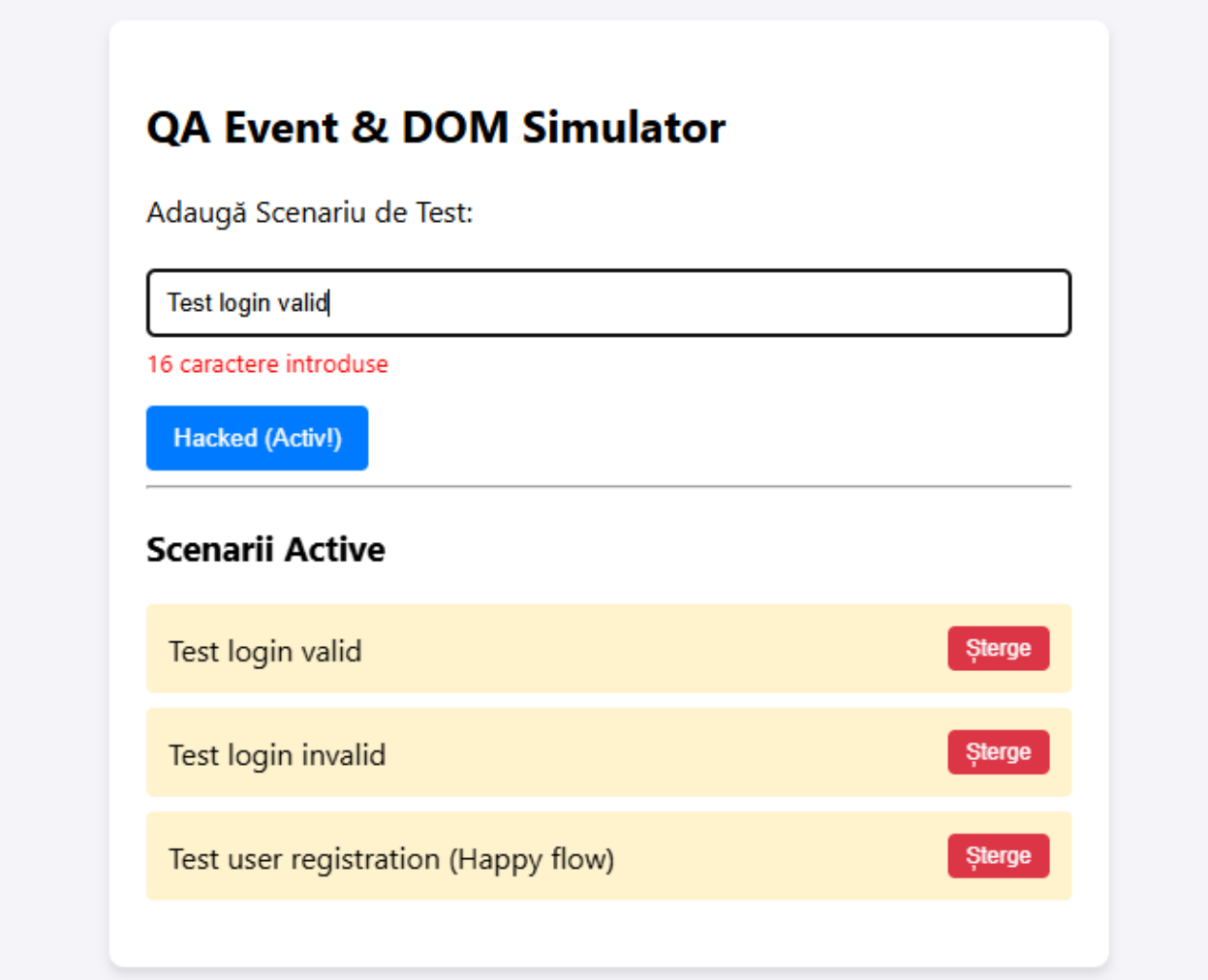
Scenariu: Vrem un avertisment vizual dacă textul introdus este prea lung (peste 15 caractere).

🤓 ❓ **Cerință**: Selectează elementul #char-count. Adaugă un ascultător input pe câmpul de text. Dacă lungimea sare de 15, fa textul numărătorului roșu.

☑ **Soluție**:

```
const contor = document.querySelector('#char-count');
const campInput = document.querySelector('#task-input');

campInput.addEventListener('input', () => {
    if (campInput.value.length > 15) {
        contor.style.color = "red";
    } else {
        contor.style.color = "gray";
    }
});
```



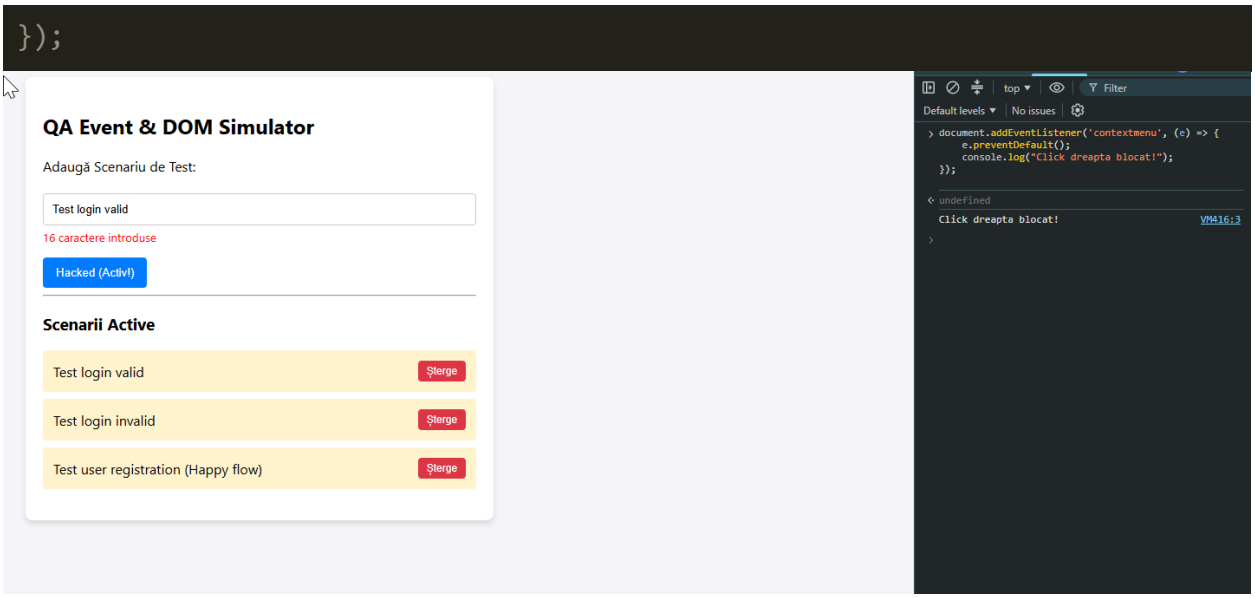
Exercițiul 6: Prevenirea acțiunilor nedorite (preventDefault)

Scenariu: Vrem să blocăm click dreapta pe pagină (meniul contextual).

🤔 ? **Cerință**: Adaugă un ascultător pe document pentru "contextmenu" și folosește e.preventDefault().

☑ **Soluție**:

```
document.addEventListener('contextmenu', (e) => {
    e.preventDefault();
    console.log("Click dreapta blocat!");
```



Secțiunea C: Manipulare Dinamică a DOM-ului

Exercițiul 7: Inserarea unui lot de date deodată

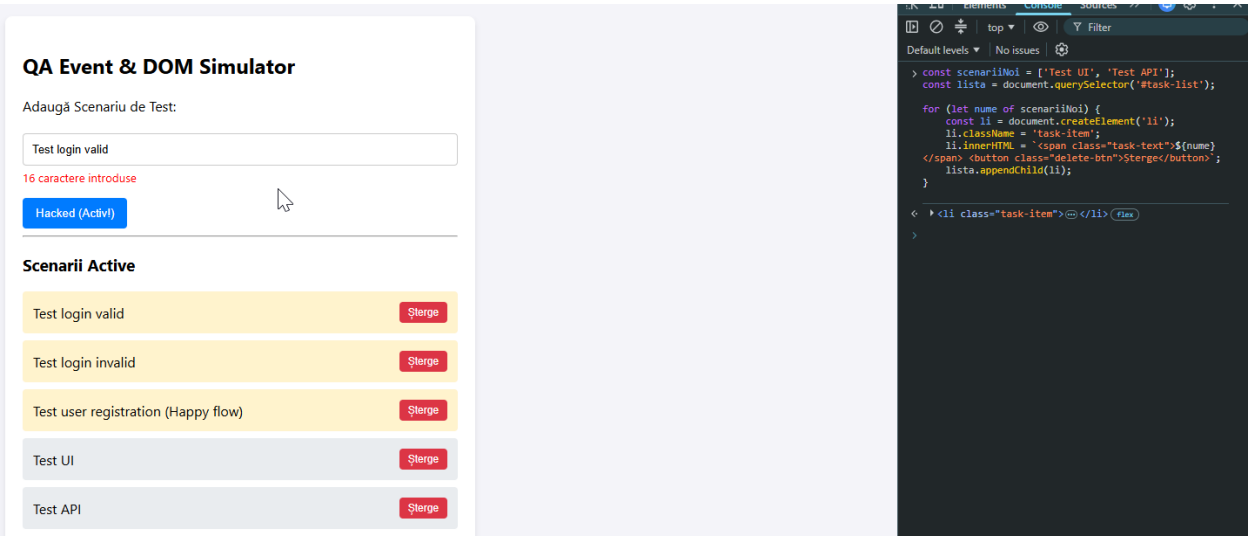
Scenariu: Randează un Array de scenarii în DOM fără click.

🤔 ❓ Cerință: Ai array-ul ['Test UI', 'Test API']. Găsește elementul #task-list. Folosește o buclă și document.createElement('li') pentru a le adăuga.

☑ **Soluție**:

```
const scenariiNoi = ['Test UI', 'Test API'];
const lista = document.querySelector('#task-list');

for (let nume of scenariiNoi) {
    const li = document.createElement('li');
    li.className = 'task-item';
    li.innerHTML = `<span class="task-text">${nume}</span> <button
class="delete-btn">Șterge</button>`;
    lista.appendChild(li);
}
```



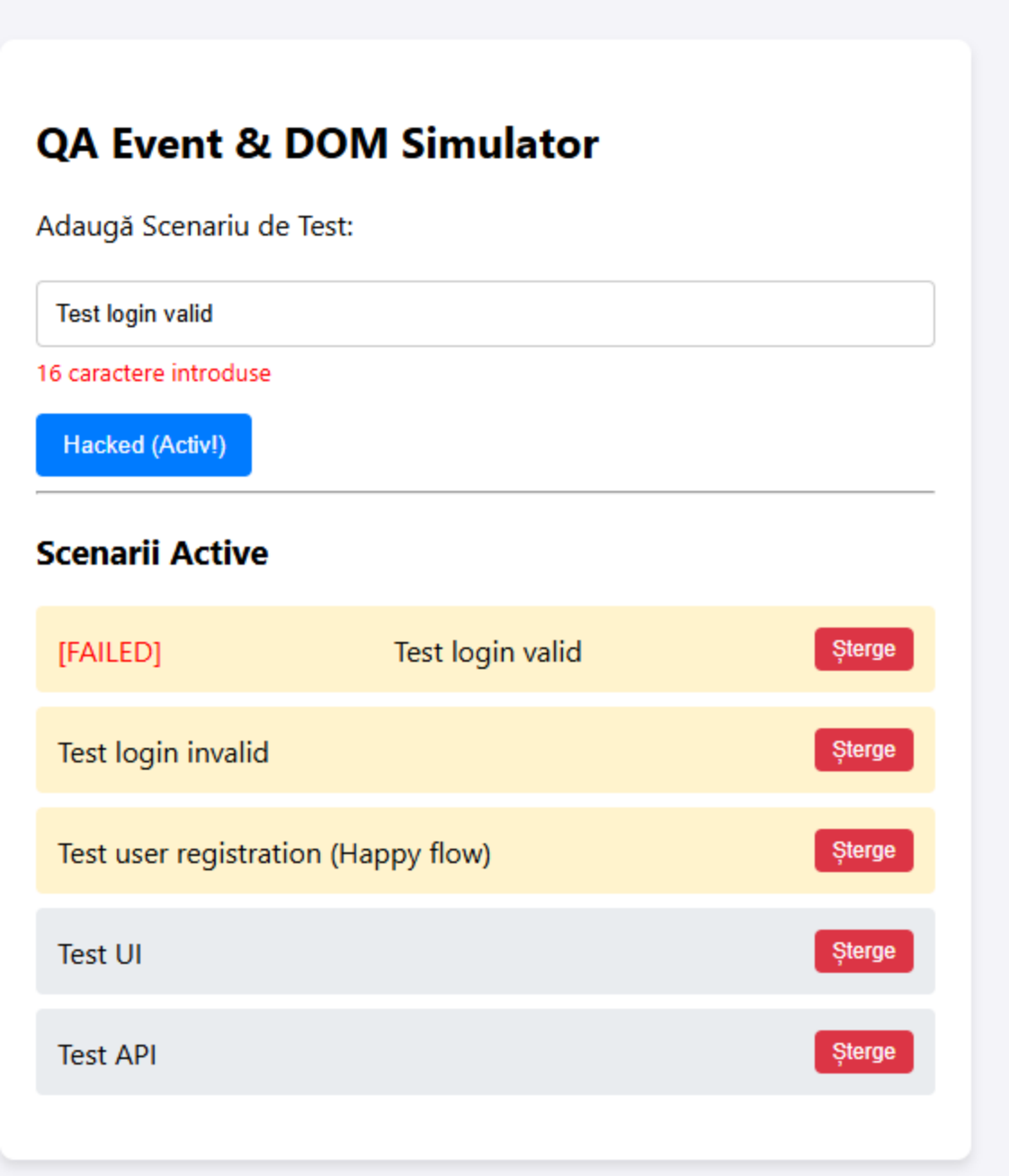
Exercițiul 8: Adăugarea unui indicator de "Test Picat"

Scenariu: Vrei să adaugi un "tag" roșu de [FAILED] lângă primul element.

🤓 ❓ **Cerință**: Găsește primul element <li> (care are clasa .task-item). Creează dinamic un tag , pune-i textul [FAILED], apoi adaugă-l folosind prepend() (care adaugă la început).

☑ **Soluție**:

```
const primulLi = document.querySelector('.task-item');
if (primulLi) {
    const failedTag = document.createElement('span');
    failedTag.textContent = "[FAILED] ";
    failedTag.style.color = "red";
    primulLi.prepend(failedTag);
}
```



Secțiunea D: Latență și Propagare Avansată (Bubbling)

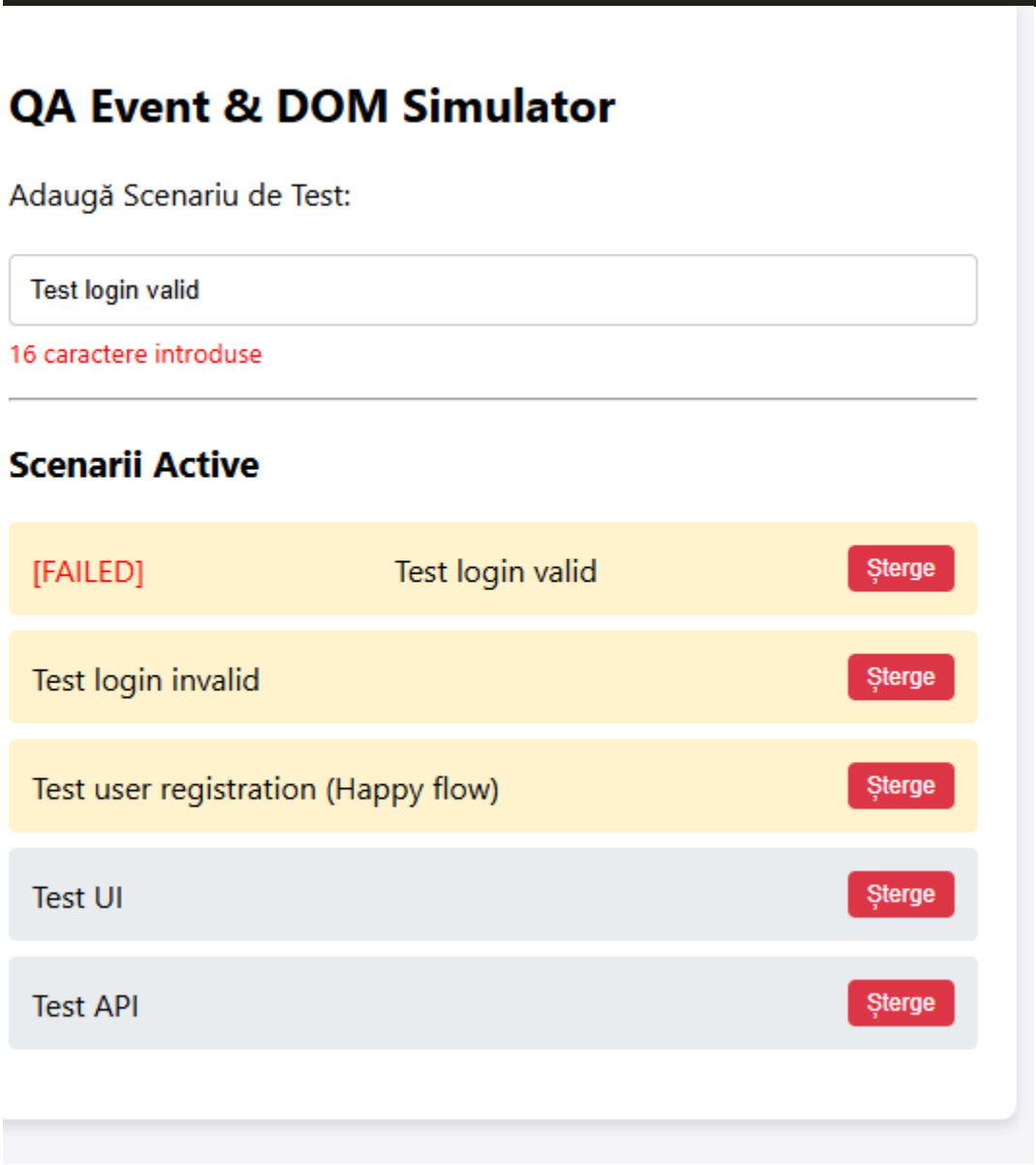
Exercițiul 9: Latența Distructivă (Testarea Flaky)

Scenariu: Butonul de submit dispare de pe ecran brusc.

🤔 ❓ **Cerință**: Găsește butonul de submit. Adaugă un timer (setTimeout). După 3000 de milisecunde, distruge complet butonul folosind remove().

☑ **Soluție**:

```
const butonulSalvare = document.querySelector('#submit-btn');
setTimeout(() => {
    butonulSalvare.remove();
}, 3000);
```



Exercițiul 10: Event Delegation Peste Tot - Bulele de Săpun (Status "Passed")

Scenariu: Vrem ca testerul să poată da click *oriunde* pe rândul unui scenariu pentru a-l marca puternic vizual ca "Passed". Folosirea lui e.target chior ne poate înșela dacă utilizatorul dă click pe marginea goală a elementului li și nu direct pe text.

🤔 ❓ **Cerință**: Adaugă un ascultător pe #task-list. Folosește e.target.closest('.task-item') pentru a prinde întotdeauna cutia rândului, indiferent unde s-a dat click. Dacă s-a dat click pe butonul de ștergere, ignoră comanda (pentru a nu bloca ștergerea). Dacă ai găsit rândul valid, schimbă-i fundalul în #d4edda și adaugă-i un border-left verde aprins pentru un UI de confirmare.

☑ **Soluție**:

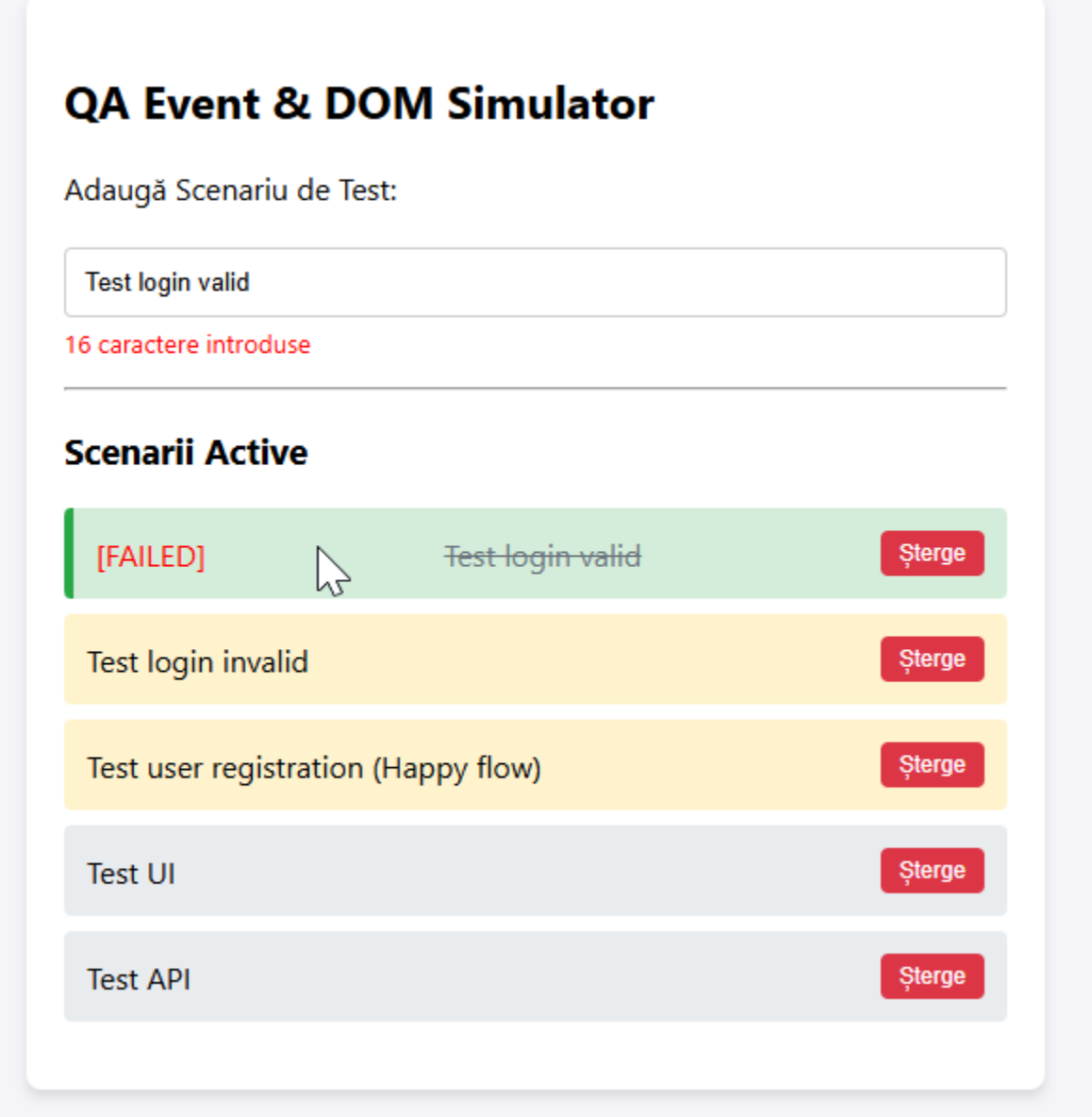
```
const listaPrincipala = document.querySelector('#task-list');

listaPrincipala.addEventListener('click', (e) => {
    // 1. Dacă click-ul a fost fix pe butonul roșu de "Șterge", ieșim
imediat (acela are altă treabă)
    if (e.target.classList.contains('delete-btn')) return;

    // 2. Căutăm cel mai apropiat element părinte care este rândul nostru
(cutia completă)
    const randApasat = e.target.closest('.task-item');

    // 3. Dacă am dat click în interiorul unui rând, îl transformăm vizual
în starea "Passed"
    if (randApasat) {
        randApasat.style.backgroundColor = '#d4edda';
        randApasat.style.borderLeft = '5px solid #28a745';

        // Opțional: putem tăia și textul cum făceam înainte, căutându-l
doar în rândul acesta
        const textSpan = randApasat.querySelector('.task-text');
        if (textSpan) {
            textSpan.classList.add('completed');
        }
    }
});
```



Partea 3: Nivelul "Masterclass" - Exerciții Avansate pentru QA

Aceste exerciții simulează exact metodele pe care Playwright le folosește intern pentru a "citi" sau "forța" starea elementelor.

Exercițiul 11: Colectarea datelor într-un Array real

(Scraping)

Scenariu: În automatizare, vrei să iei textul tuturor celor 5 produse de pe ecran și să verifici dacă sunt sortate alfabetic.

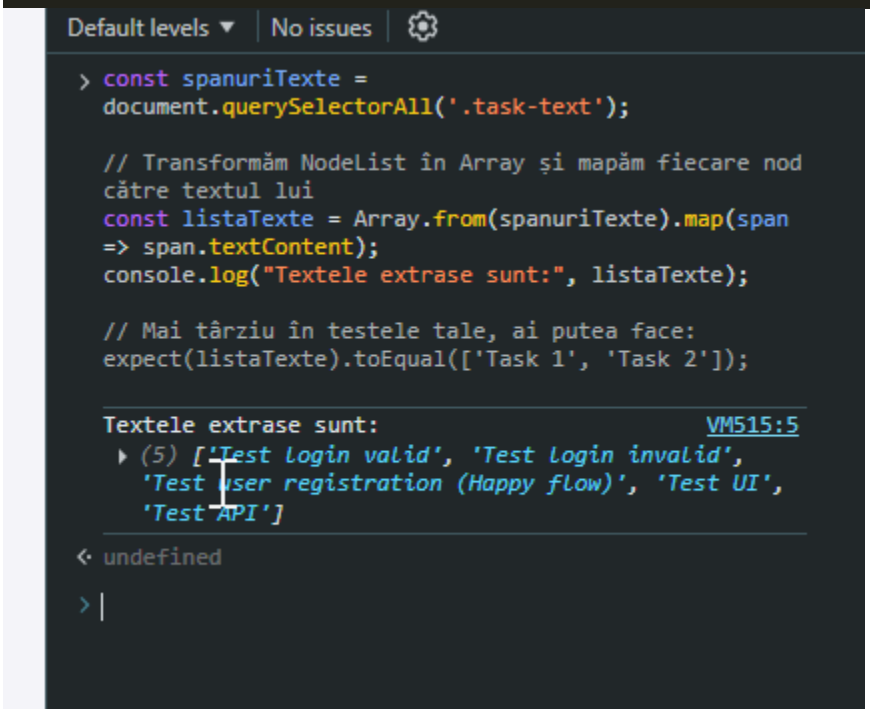
😳 ❓ **Cerință**: Ai adăugat 3 task-uri pe ecran. querySelectorAll returnează un NodeList, dar tu vrei un Array simplu doar cu textele. Găsește toate .task-text, folosește Array.from() și metoda .map() pentru a extrage proprietatea textContent într-o listă nouă.

☑ **Soluție**:

```
const spanuriTexte = document.querySelectorAll('.task-text');

// Transformăm NodeList în Array și mapăm fiecare nod către textul lui
const listaTexte = Array.from(spanuriTexte).map(span => span.textContent);
console.log("Textele extrase sunt:", listaTexte);

// Mai târziu în testele tale, ai putea face:
expect(listaTexte).toEqual(['Task 1', 'Task 2']);
```



Exercițiul 12: Navigarea Relativă în DOM (Element

Matching)

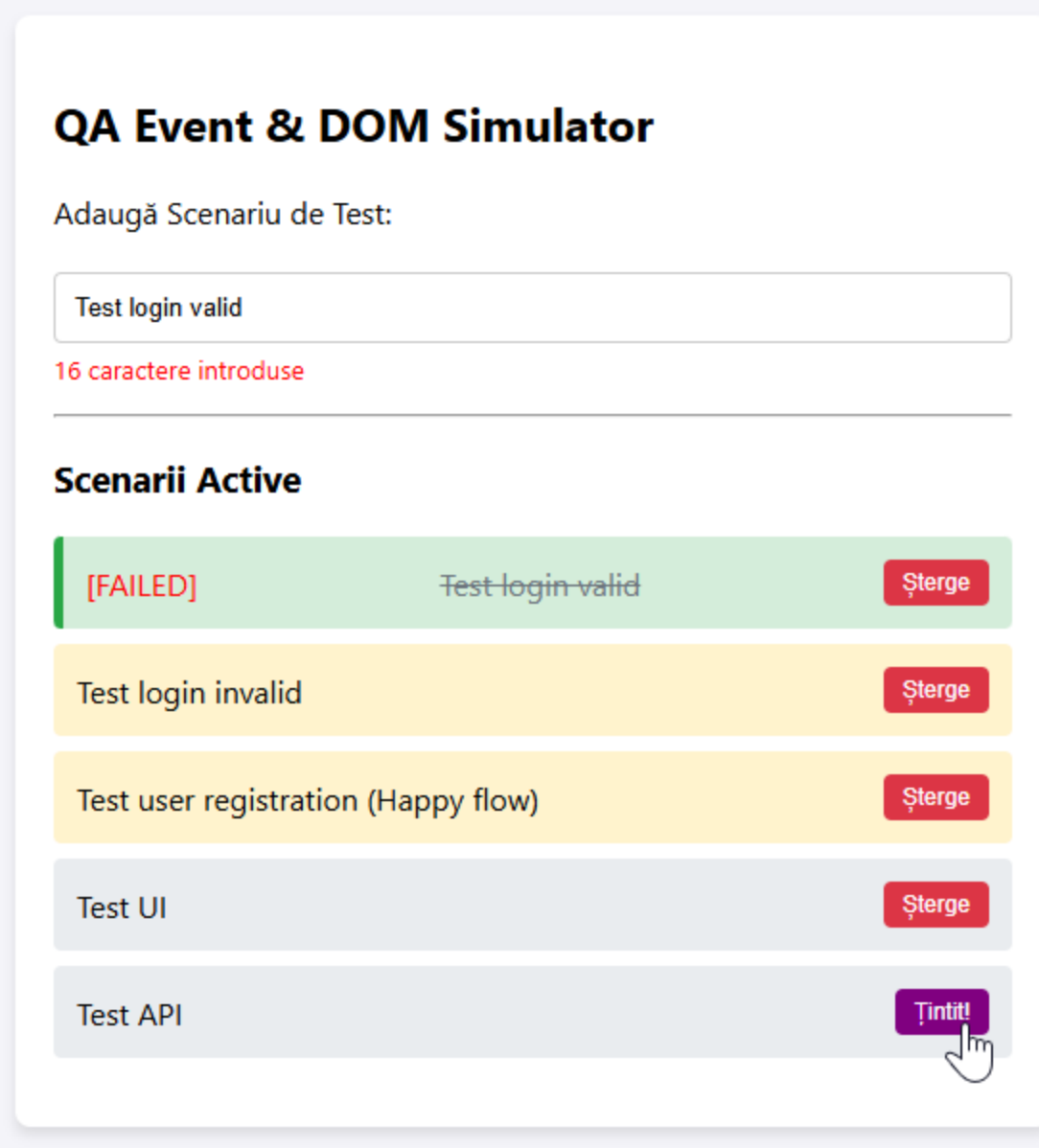
Scenariu: Ai un tabel cu zeci de produse. Vrei să dai click pe butonul de "Ștergere" care aparține doar task-ului cu numele "Test API" (fără a șterge altceva).

😳 ❓ **Cerință**: Găsește tag-ul care conține textul "Test API" (va trebui să iterezi prin ele). Odată găsit, folosește proprietatea nextElementSibling pentru a accesa direct butonul de lângă el și a-l colora în roșu aprins.

☑ **Soluție**:

```
const toateSpanurile = document.querySelectorAll('.task-text');

for (let span of toateSpanurile) {
    if (span.textContent === 'Test API') {
        // Am găsit textul! Acum navigăm la elementul frate (butonul de
lângă el)
        const butonulLui = span.nextElementSibling;
        butonulLui.style.backgroundColor = "purple";
        butonulLui.textContent = "Țintit!";
        break;
    }
}
```



Exercițiul 13: Declanșarea Programatică a Evenimentelor (Bypass)

Scenariu: Uneori, completarea unui câmp de text dintr-un script JavaScript (ex: input.value = 'Hacked') nu activează butoanele, deoarece framework-ul UI (precum React) nu detectează "evenimentul" uman de tastare.

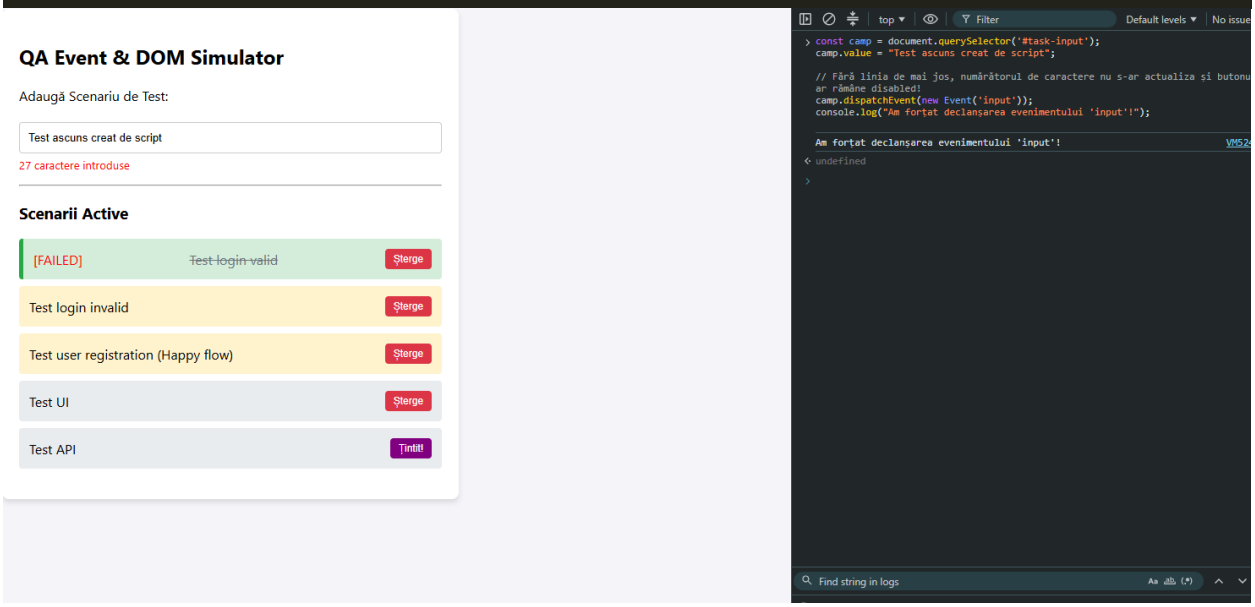
🤯 ❓ **Cerință**: Setează valoarea input-ului la "Test ascuns". Apoi, folosește metoda dispatchEvent(new Event('input')) pe acel input pentru a minți browserul că un om a tastat fizic

acolo, declanșând validările.

☑ **Soluție**:

```
const camp = document.querySelector('#task-input');
camp.value = "Test ascuns creat de script";

// Fără linia de mai jos, numărătorul de caractere nu s-ar actualiza și
butonul ar rămâne disabled!
camp.dispatchEvent(new Event('input'));
console.log("Am forțat declanșarea evenimentului 'input'!");
```



Exercițiul 14: Simulare Checkbox "Terms & Conditions"

Scenariu: Vrei să adaugi o constrângere nouă: formularul nu se poate trimite decât dacă un "Checkbox" este bifat.

🤓 ❓ **Cerință**: Creează dinamic un element <input type="checkbox"> și un <label> cu textul "Sunt de acord". Inserează-le fix înainte de butonul de submit (folosind form.insertBefore). Apoi adaugă o condiție: butonul devine activ DOAR dacă checkbox-ul este bifat (checkbox.checked === true) ȘI input-ul are text.

☑ **Soluție**:

```
// 1. Creăm elementele
const formularul = document.querySelector('#qa-form');
const btn = document.querySelector('#submit-btn');
const inputPrincipal = document.querySelector('#task-input');

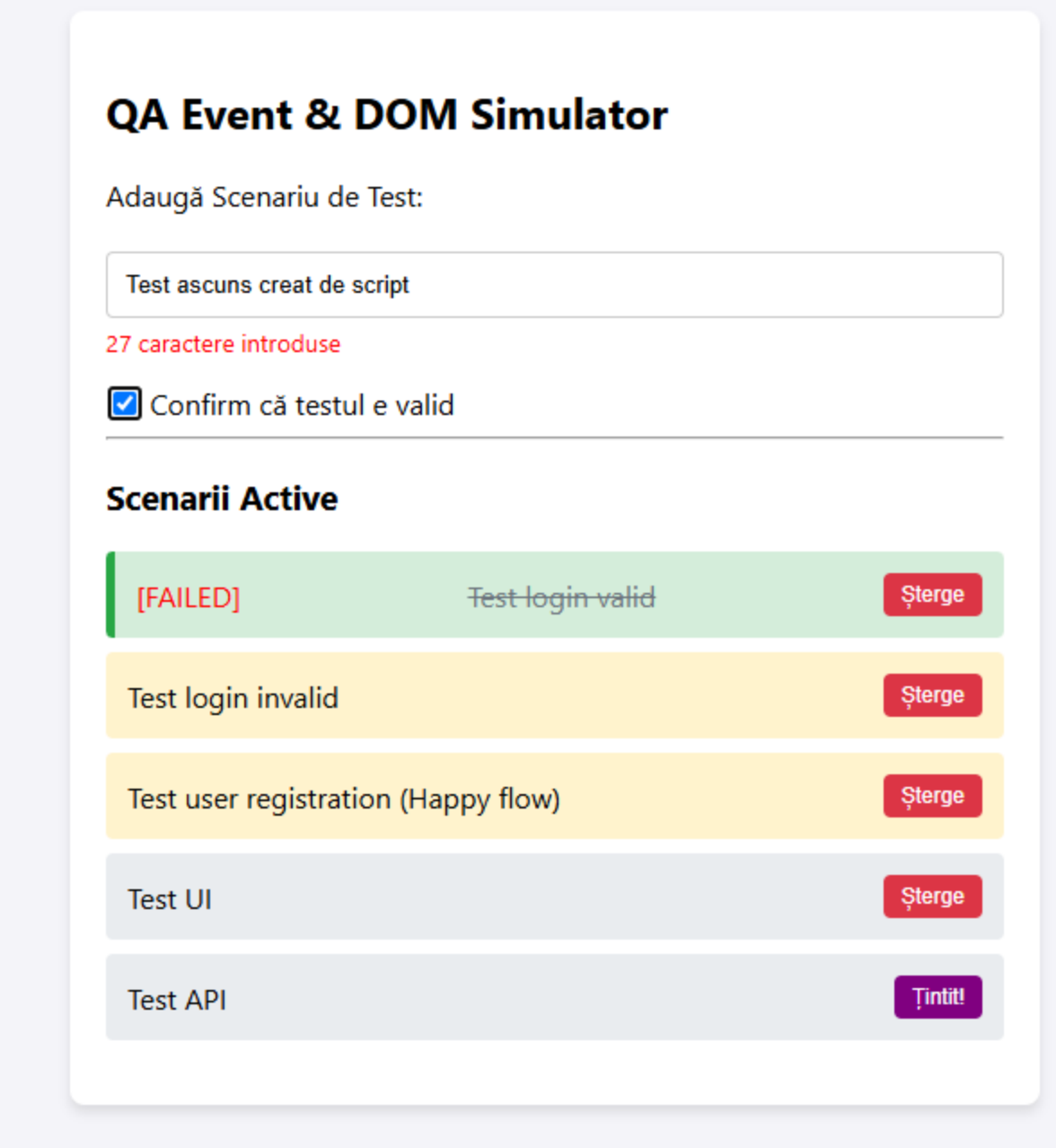
const checkbox = document.createElement('input');
checkbox.type = 'checkbox';
checkbox.id = 'terms';

const label = document.createElement('label');
label.textContent = " Confirm că testul e valid ";

// 2. Le inserăm în DOM înainte de buton
formularul.insertBefore(checkbox, btn);
formularul.insertBefore(label, btn);

// 3. Modificăm logica (ascultăm pe ambele)
function valideazaFormular() {
    if (inputPrincipal.value.length > 0 && checkbox.checked) {
        btn.disabled = false;
    } else {
        btn.disabled = true;
    }
}

// Atașăm validarea la ambele evenimente
inputPrincipal.addEventListener('input', valideazaFormular);
checkbox.addEventListener('change', valideazaFormular);
```



Exercițiul 15: Validarea vizibilității (Computed Style)

Scenariu: Playwright verifică des dacă un element este cu adevărat vizibil pe ecran (nu ascuns cu CSS display: none).

🤔 ❓ **Cerință**: Elementul #loading-msg apare și dispare. Scrie o funcție care preia stilul real

calculat al elementului folosind metoda window.getComputedStyle(element). Verifică dacă proprietatea display este "none". Afișează în consolă: "Elementul este ASCUNS" sau "Elementul este VIZIBIL".

☑ **Soluție**:

```
const mesajLoading = document.querySelector('#loading-msg');

// getComputedStyle aduce stilurile reale, calculate la momentul execuției
const stilCalculat = window.getComputedStyle(mesajLoading);

if (stilCalculat.display === 'none') {
    console.log("Asertare trecută: Elementul este ASCUNS pe ecran.");
} else {
    console.log("Asertare picată: Elementul este încă VIZIBIL pe ecran!");
}
```

